

查询千万数据避免内存溢出

由于现在 ORM 框架的成熟运用，很多小伙伴对于 JDBC 的概念有些薄弱，**ORM 框架底层其实是通过 JDBC 操作的 DB。**

JDBC (JavaDataBase Connectivity) 是 Java 数据库连接, 说的直白点就是使用 **Java 语言操作数据库。**

由 **SUN 公司**提供出一套访问数据库的规范 **API**，并提供相对应的连接数据库协议标准，然后各厂商根据规范提供一套访问自家数据库的 **API 接口。**

MySql JDBC 大数据量操作

整篇文章以大数据量操作为议题，通过开发过程中的需求引出相关知识点：

1. 迁移数据
2. 导出数据
3. 批量处理数据

一般而言笔者认为在 Java Web 程序里，能够被称为大数据量的，几十万到千万不等，再高的话 Java (WEB 应用) 处理就不怎么合适了。

举个例子，现在业务系统需要从 MySQL 数据库里读取 500w 数据行进行处理，应该怎么做：

1. 常规查询，一次性读取 500w 数据到 JVM 内存中，或者分页读取。
2. 流式查询，建立长连接，利用服务端游标，每次读取一条加载到 JVM 内存。
3. 游标查询，和流式一样，通过 fetchSize 参数，控制一次读取多少条数据。

1. 常规查询

默认情况下，完整的检索结果集会将其存储在内存中。在大多数情况下，这是最有效的操作方式，并且由于 MySQL 网络协议的设计，因此更易于实现。

假设单表 500w 数据量，没有人会一次性加载到内存中，一般会采用分页的方式。

```

@SneakyThrows
@Override
public void pageQuery() {
    @Cleanup Connection conn = dataSource.getConnection();
    @Cleanup Statement stmt = conn.createStatement();
    long start = System.currentTimeMillis();
    long offset = 0;
    int size = 100;
    while (true) {
        String sql = String.format("SELECT COLUMN_A, COLUMN_B, COLUMN_C FROM YOU_TABLE
LIMIT %s, %s", offset, size);
        @Cleanup ResultSet rs = stmt.executeQuery(sql);
        long count = loopResultSet(rs);
        if (count == 0) break;
        offset += size;
    }

    log.info(" 🚀🚀🚀 分页查询耗时 :: {} ", System.currentTimeMillis() - start);
}

```

上述方式比较简单，但是在不考虑 LIMIT 深分页优化情况下，线上数据库服务器就凉了，**亦或者你能等个几天时间检索数据。**

2. 流式查询

如果你正在使用具有大量数据行的 ResultSet，并且无法在 JVM 中为其分配所需的内存堆空间，则可以告诉驱动程序从结果流中返回一行。

The combination of a forward-only, read-only result set, with a fetch size of `Integer.MIN_VALUE` serves as a signal to the driver to stream result sets row-by-row. After this, any result sets created with the statement will be retrieved row-by-row.

There are some caveats with this approach. You must read all of the rows in the result set (or close it) before you can issue any other queries on the connection, or an exception will be thrown.

The earliest the locks these statements hold can be released (whether they be `MyISAM` table-level locks or row-level locks in some other storage engine such as `InnoDB`) is when the statement completes.

If the statement is within scope of a transaction, then locks are released when the transaction completes (which implies that the statement needs to complete first). As with most other databases, statements are not complete until all the results pending on the statement are read or the active result set for the statement is closed.

Therefore, if using streaming results, process them as quickly as possible if you want to maintain concurrent access to the tables referenced by the statement producing the result set.

流式查询有一点需要注意：**必须先读取（或关闭）结果集中的所有行**，然后才能对连接发出任何其他查询，否则将引发异常。

使用流式查询，则要保持对产生结果集的语句所引用的表的并发访问，因为其 **查询会独占连接，所以必须尽快处理**。

```
@SneakyThrows
public void streamQuery() {
    @Cleanup Connection conn = dataSource.getConnection();
    @Cleanup Statement stmt = conn.createStatement(ResultSet.TYPE_FORWARD_ONLY,
        ResultSet.CONCUR_READ_ONLY);
    stmt.setFetchSize(Integer.MIN_VALUE);

    long start = System.currentTimeMillis();
    @Cleanup ResultSet rs = stmt.executeQuery("SELECT COLUMN_A, COLUMN_B, COLUMN_C FROM
        YOU_TABLE");
    loopResultSet(rs);
    log.info(" 🚀🚀🚀 流式查询耗时 :: {} ", (System.currentTimeMillis() - start) /
        1000);
}
```

流式查询库表数据量 500w 单次调用时间消耗：≈ 6s。

3. 游标查询

SpringBoot 2.x 版本默认连接池为 HikariPool，连接对象是 HikariProxyConnection，所以下述设置游标方式就不可行了。

```
((JDBC4Connection) conn).setUseCursorFetch(true);
```

需要在数据库连接信息里拼接 **&useCursorFetch=true**。其次设置 Statement 每次读取数据数量，比如一次读取 1000。

```
@SneakyThrows
public void cursorQuery() {
    @Cleanup Connection conn = dataSource.getConnection();
    @Cleanup Statement stmt = conn.createStatement(ResultSet.TYPE_FORWARD_ONLY,
        ResultSet.CONCUR_READ_ONLY);
```

```

stmt.setFetchSize(1000);

long start = System.currentTimeMillis();
@Cleanup ResultSet rs = stmt.executeQuery("SELECT COLUMN_A, COLUMN_B, COLUMN_C FROM
YOU_TABLE");
loopResultSet(rs);
log.info(" 🚀🚀🚀 游标查询耗时 :: {} ", (System.currentTimeMillis() - start) /
1000);
}

```

游标查询库表数据量 500w 单次调用时间消耗：≈ 18s。

4. JDBC RowData

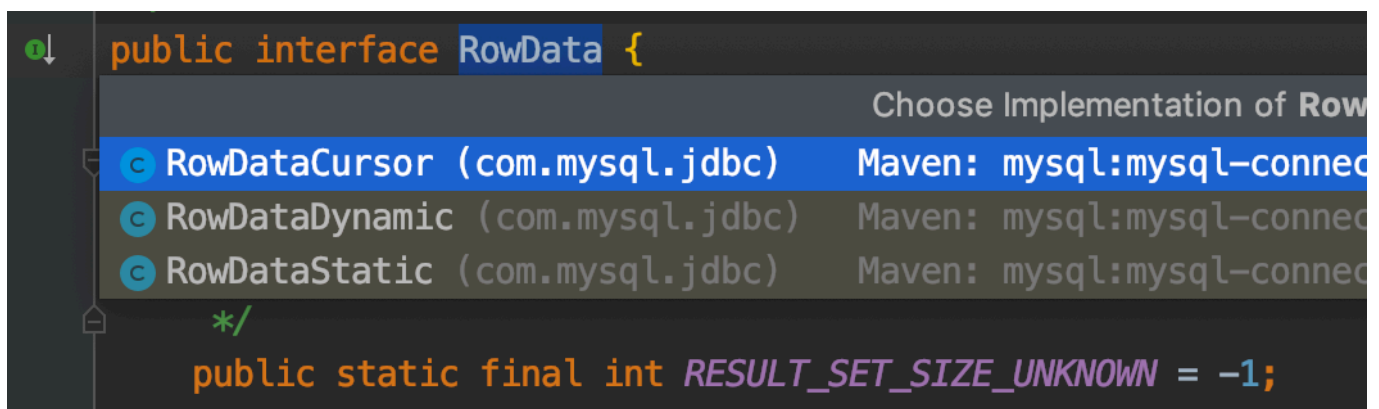
上面都使用到了方法 loopResultSet，方法内部只是进行了 while 循环，常规、流式、游标查询的核心点在于 next 方法。

```

@sneakyThrows
private Long loopResultSet(ResultSet rs) {
    while (rs.next()) {
        // 业务操作
    }
    return xx;
}

```

ResultSet.next() 的逻辑是实现类 ResultSetImpl 每次都从 RowData 获取下一行的数据。RowData 是一个接口，实现关系图如下。



默认情况下 ResultSet 会使用 RowDataStatic 实例，在生成 RowDataStatic 对象时就会把 ResultSet 中所有记录读到内存里，之后通过 next() 再一条条从内存中读。

RowDataCursor 的调用为批处理，然后进行内部缓存，流程如下：

1. 首先会查看自己内部缓冲区是否有数据没有返回，如果有则返回下一行。
2. 如果都读取完毕，向 MySQL Server 触发一个新的请求读取 fetchSize 数量结果。
3. 并将返回结果缓冲到内部缓冲区，然后返回第一行数据。

当采用流式处理时，**ResultSet 使用的是 RowDataDynamic 对象**，而这个对象 next() 每次调用都会发起 IO 读取单行数据。

总结来说就是，**默认的 RowDataStatic 读取全部数据到客户端内存中**，也就是我们的 JVM；**RowDataCursor 一次读取 fetchSize 行**，消费完成再发起请求调用；**RowDataDynamic 每次 IO 调用读取一条数据**。

5. JDBC 通信原理

5.1 普通查询

在 JDBC 与 MySQL 服务端的交互是通过 Socket 完成的，对应到网络编程，可以把 **MySQL 当作一个 SocketServer**，因此一个完整的请求链路应该是：

JDBC 客户端 -> 客户端 Socket -> MySQL -> 检索数据返回 -> MySQL 内核 Socket 缓冲区 -> 网络 -> 客户端 Socket Buffer -> JDBC 客户端。

普通查询的方式在查询大数据量时，所在 JVM 可能会凉凉，原因如下：

1. MySQL Server 会将检索出的 SQL 结果集通过输出流写入到内核对应的 Socket Buffer。
2. **内核缓冲区通过 JDBC 发起的 TCP 链路进行回传数据**，此时数据会先进入 JDBC 客户端所在内核缓冲区。
3. JDBC 发起 SQL 操作后，程序会被阻塞在输入流的 read 操作上，当缓冲区有数据时，程序会被唤醒进而将缓冲区数据读取到 JVM 内存中。
4. MySQL Server 会不断发送数据，JDBC 不断读取缓冲区数据到 Java 内存中，**虽然此时数据已到 JDBC 所在程序本地，但是 JDBC 还没有对 execute 方法调用处进行响应**，因为需要等

到对应数据读取完毕才会返回。

5. 弊端就显而易见了，如果查询数据量过大，会不断经历 GC，然后就是内存溢出。

5.2 游标查询

通过上文得知，游标可以解决普通查询大数据量的内存溢出问题，但是。

小伙伴有没有思考过这么一个问题，**MySQL 不知道客户端程序何时消费完成，此时另一连接对该表造成 DML 写入操作应该如何处理？**

其实，在我们使用游标查询时，MySQL 需要建立一个临时空间来存放需要被读取的数据，**所以不会和 DML 写入操作产生冲突。**

但是游标查询会引发以下现象：

1. **IOPS 飙升**，因为需要返回的数据需要写入到临时空间中，**存在大量的 IO 读取和写入**，此流程可能会引起其它业务的写入抖动。
2. **磁盘空间飙升**，因为写入临时空间的数据是在原表之外的，如果表数据过大，**极端情况下可能会导致数据库磁盘写满**，这时网络输出时没有变化的。而写入临时空间的数据会在**读取完成或客户端发起 ResultSet#close 操作时由 MySQL 回收。**
3. 客户端 JDBC 发起 SQL 查询，可能会有长时间等待 SQL 响应，这段时间为服务端准备数据阶段。但是**普通查询等待时间与游标查询等待时间原理上是不一致的**，前者是一致在读取网络缓冲区的数据，没有响应到业务层面；后者是 MySQL 在准备临时数据空间，没有响应到 JDBC。
4. 数据准备完成后，进行到传输数据阶段，**网络响应开始飙升，IOPS 由"读写"转变为"读取"。**

采用游标查询的方式**通信效率比较低**，因为客户端消费完 fetchSize 行数据，就需要发起请求到服务端请求，在数据库前期准备阶段**IOPS 会非常高，占用大量的磁盘空间以及性能。**

5.3 流式查询

当客户端与 MySQL Server 端建立起连接并且交互查询时，MySQL Server 会通过输出流将 SQL 结果集返回输出，也就是**向本地的内核对应的 Socket Buffer 中写入数据**，然后将内核中的数据通过 TCP 链路回传数据到 JDBC 对应的服务器内核缓冲区。

1. JDBC 通过输入流 read 方法去读取内核缓冲区数据，因为开启了流式读取，每次业务程序接收到的数据只有一条。
2. MySQL 服务端会向 JDBC 代表的客户端内核源源不断的输送数据，直到客户端请求 Socket 缓冲区满，这时的 MySQL 服务端会阻塞。
3. 对于 JDBC 客户端而言，数据每次读取都是从本机器的内核缓冲区，所以性能会更快一些，一般情况不必担心本机内核无数据消费（除非 MySQL 服务端传递来的数据，在客户端不做任何业务逻辑，拿到数据直接放弃，会发生客户端消费比服务端超前的情况）。

看起来，流式要比游标的方式更好一些，**但是事情往往不像表面上那么简单。**

1. 相对于游标查询，**流式对数据库的影响时间要更长一些。**
2. 另外流式查询依赖网络，**导致网络拥塞可能性较大。**

流式游标内存分析

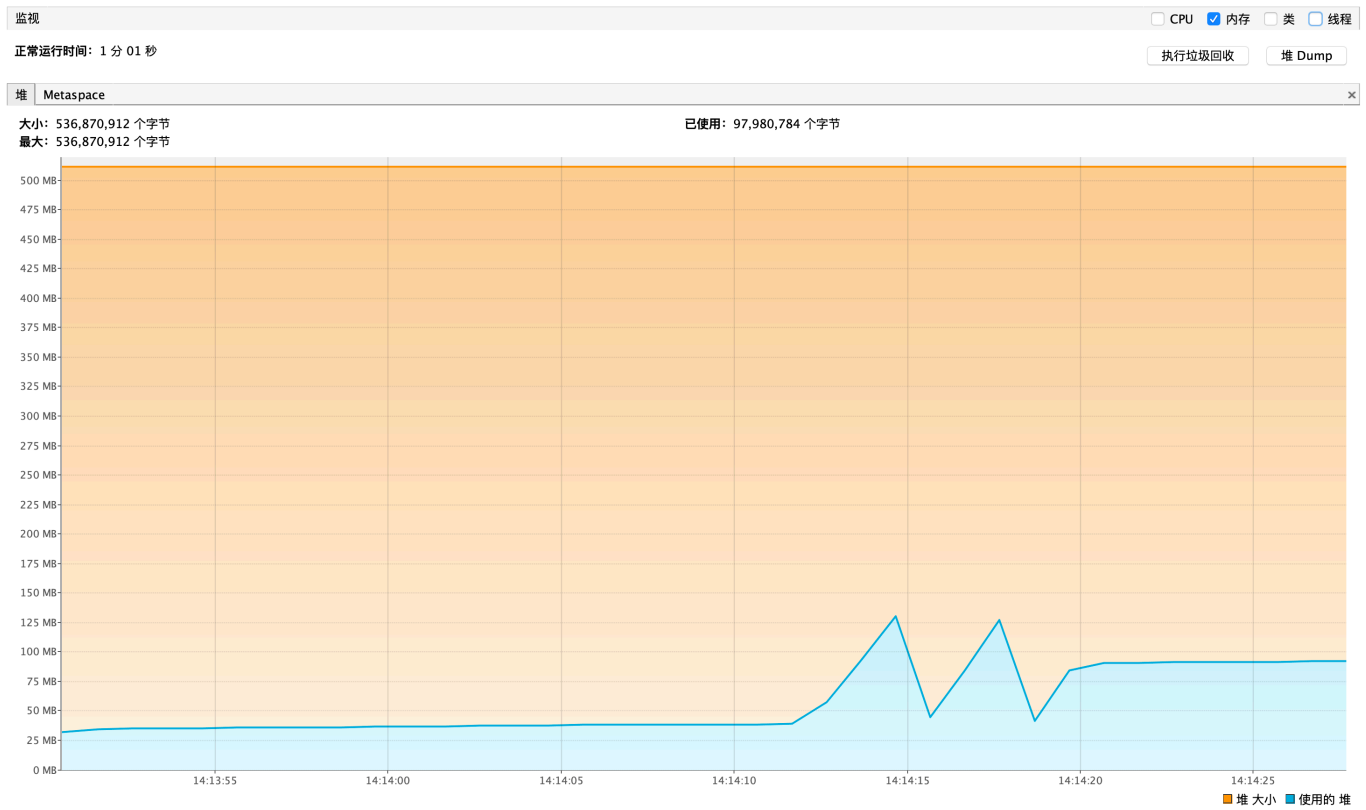
表数据量：500w。

内存查看工具：JDK 自带 Jvisualvm。

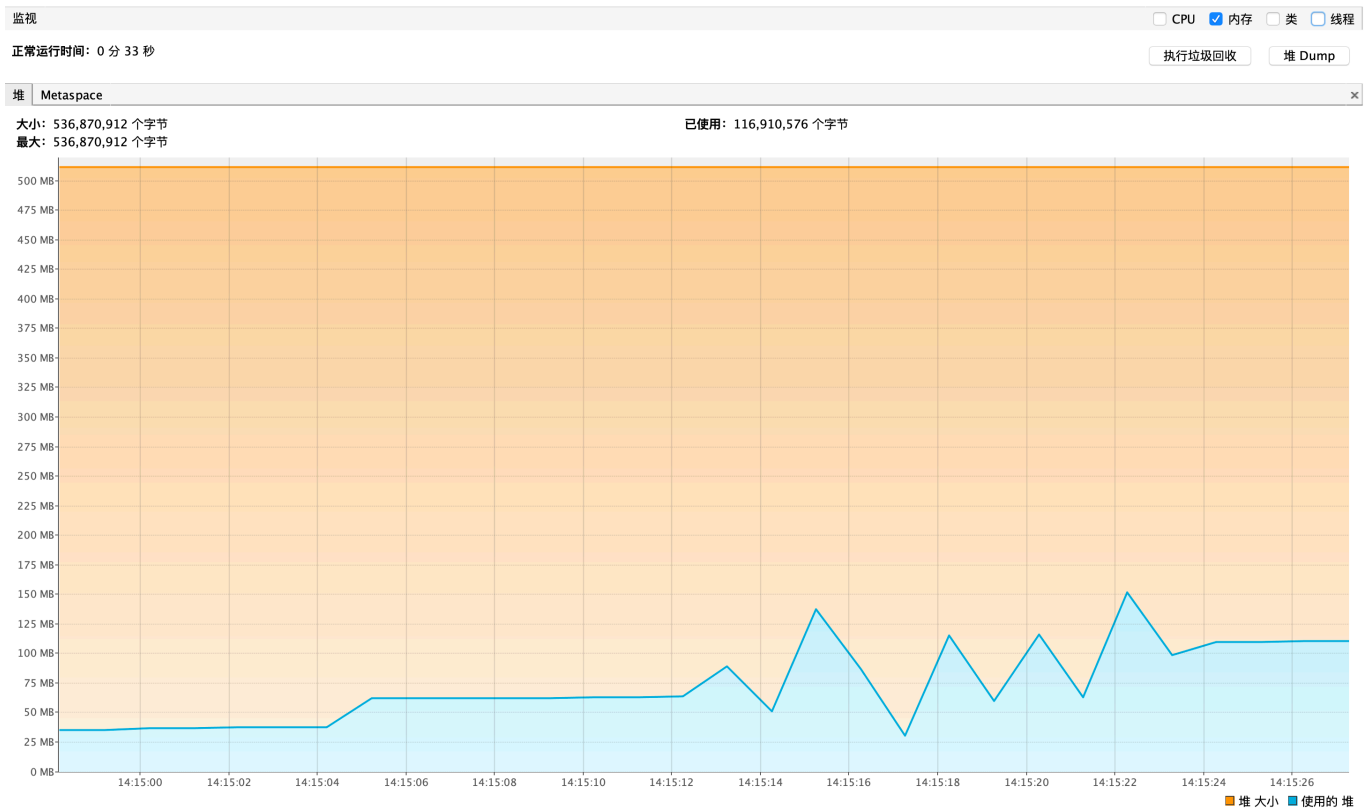
设置 JVM 参数：-Xmx512m -Xms512m。

1. 单次调用内存使用

流式查询内存性能报告如下：



游标查询内存性能报告如下：

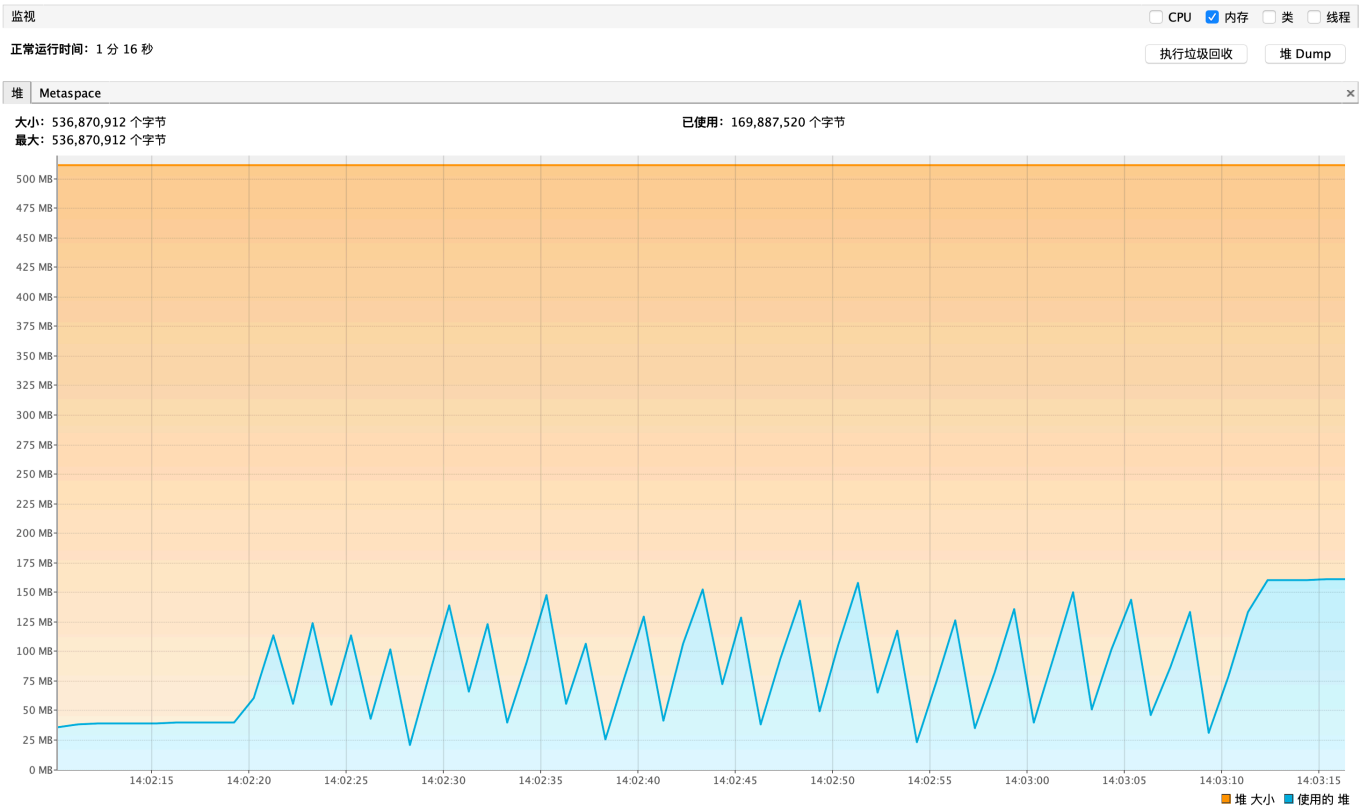


根据内存占用情况来看，游标查询和流式查询都能够很好的防止 OOM。

2. 并发调用内存使用

并发调用：Jmeter 1 秒 10 个线程并发调用。

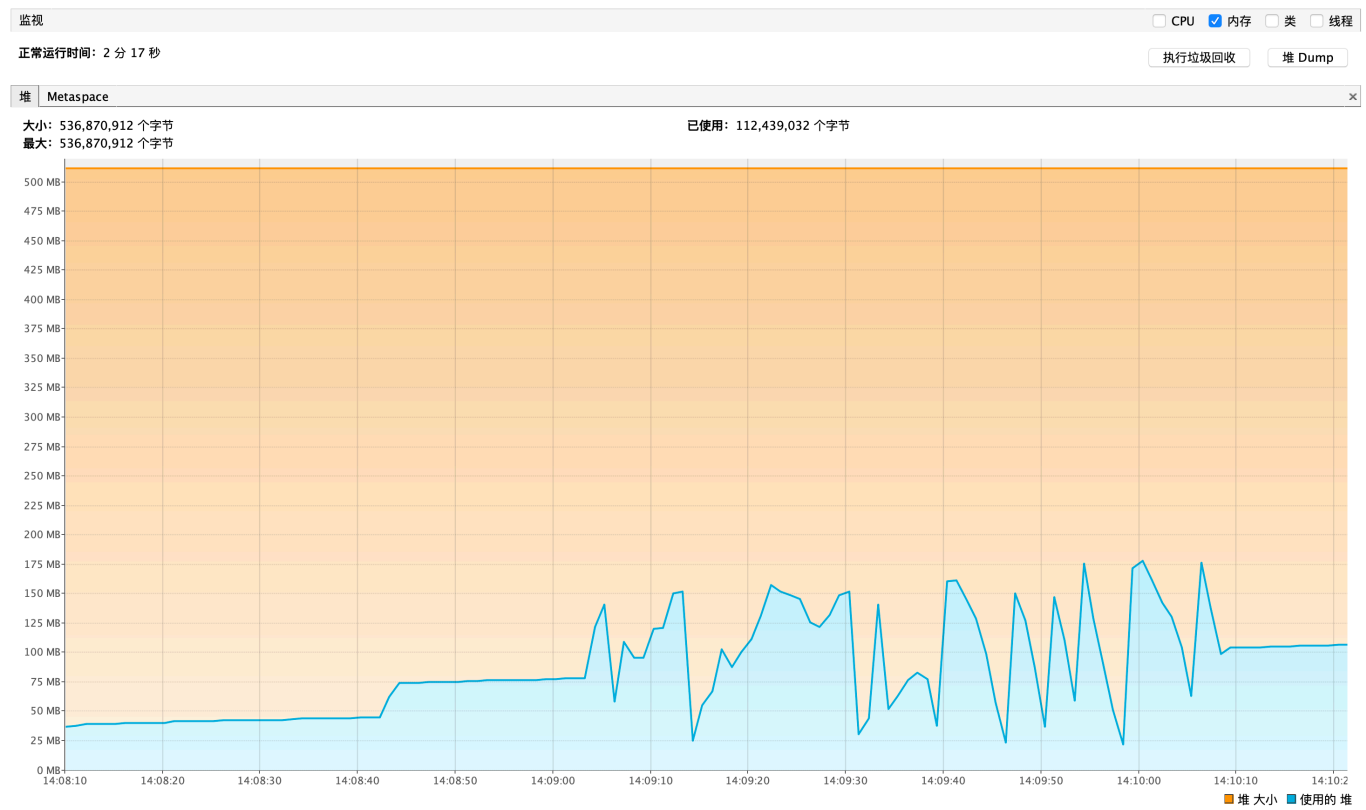
流式查询内存性能报告如下：



并发调用对于内存占用情况也很 OK，**不存在叠加式增加。**

流式查询并发调用时间平均消耗： $\approx 55s$ 。

游标查询内存性能报告如下：



游标查询并发调用时间平均消耗： $\approx 83s$ 。

因为设备限制，以及部分情况只会在极端下产生，所以没有进行生产、测试多环境验证，小伙伴感兴趣可以自行测试。

MyBatis 如何使用流式查询

上文都是在描述如何使用 JDBC 原生 API 进行查询，ORM 框架 Mybatis 也针对流式查询进行了封装。

ResultHandler 接口只包含 handleResult 方法，可以获取到已转换后的 Java 实体类。

```
@Slf4j
@Service
public class MyBatisStreamService {
    @Resource
    private MyBatisStreamMapper myBatisStreamMapper;

    public void mybatisStreamQuery() {
        long start = System.currentTimeMillis();
```

```

        myBatisStreamMapper.mybatisStreamQuery(new ResultHandler<YOU_TABLE_DO>() {
            @Override
            public void handleResult(ResultContext<? extends YOU_TABLE_DO> resultContext)
        { }

        });
        log.info(" 🚀🚀🚀 MyBatis查询耗时 :: {} ", System.currentTimeMillis() - start);
    }
}

```

除了下述注解式的应用方式，也可以使用 .xml 文件的形式。

```

@Mapper
public interface MyBatisStreamMapper {
    @Options(resultSetType = ResultSetType.FORWARD_ONLY, fetchSize = Integer.MIN_VALUE)
    @ResultType(YOU_TABLE_DO.class)
    @Select("SELECT COLUMN_A, COLUMN_B, COLUMN_C FROM YOU_TABLE")
    void mybatisStreamQuery(ResultHandler<YOU_TABLE_DO> handler);
}

```

Mybatis 流式查询调用时间消耗：≈ **18s**。

JDBC 流式与 MyBatis 封装的流式读取对比：

1. MyBatis 相对于原生的流式还是慢上了不少，但是考虑到底层的封装的特性，这点性能还是可以接受的。
2. 从内存占比而言，两者波动相差无几。
3. MyBatis 相对于原生 JDBC 更为的方便，因为封装了回调函数以及序列化对象等特性。

两者具体的使用，可以针对项目实际情况而定，**没有最好的，只有最适合的。**

文末结语

流式查询、游标查询可以避免 OOM，**数据量大可以考虑此方案**。但是这两种方式会占用数据库连接，使用中不会释放，所以线上针对大数据量业务用到游标和流式操作，**一定要进行并发控制**。

另外针对 JDBC 原生流式查询，Mybatis 中也进行了封装，虽然会慢一些，但是 **功能以及代码的整洁程度会好上不少**。

参考文章:

1. <https://blog.csdn.net/xieyuooo/article/details/83109971>

更新: 2023-07-17 18:38:00

原文: <<https://www.yuque.com/magestack/12306/tfvph41m9b6k0grz>>